Methodical Guide Variant

Methodical Guide

Сначала восстановим постановку задачи для твоего варианта, а потом перейдём к простым объяснениям и опорным графикам.

Формальный отчёт нужен для сдачи. Это руководство нужно, чтобы быстро понять смысл расчётов и безопасно проговорить их на защите.

Все числа ниже относятся только к текущему варианту и итоговым расчётам. Для удобства чтения десятичные значения местами округлены до 3-4 знаков после запятой.

Постановка задачи для твоего варианта

Задача 1. Что именно задано

В задаче задан колл-центр со средним временем между звонками `Tc` и средним временем обслуживания `Ts`. Потоки считаются пуассоновскими, а дальше нужно отдельно рассмотреть четыре режима одной и той же системы:

- без очереди
- с ограниченной очередью
- без ограничений на длину очереди
- без ограничений на длину очереди с уходом клиентов

Для твоего варианта это означает:

- `Tc = 14` с
- `Ts = 65` с
- `Tw = 106` с для подпункта `1.4`

Для расчёта сразу используем:

- `λ = 1 / Tc = 0.0714` 1/с
- `μ = 1 / Ts = 0.0154` 1/с
- `ν = 1 / Tw = 0.00943` 1/с для подпункта `1.4`
- `a = λ / μ = 4.643`

Коротко про обозначения:

- `Tc` - среднее время между звонками
- `Ts` - среднее время обслуживания одного звонка
- `Tw` - среднее приемлемое время ожидания в очереди в модели с уходом клиентов
- `λ` - интенсивность поступления звонков
- `μ` - интенсивность обслуживания одним оператором
- `ν` - интенсивность ухода клиента из очереди
- `a` - приведённая нагрузка потока

Задача 2. Что именно задано

В задаче задан участок с `N` станками, средним временем между наладками `Tc` и средним временем наладки `Ts`. Дальше нужно посмотреть, как при изменении числа наладчиков меняются простои, ожидание и занятость.

Для твоего варианта это означает:

- `N = 36` станков
- `Tc = 104` мин
- `Ts = 50` мин

Для расчёта сразу используем:

- λ `= 1 / Tc = 0.00962` 1/мин
- μ `= 1 / Ts = 0.0200` 1/мин

Коротко про обозначения:

- `N` - число станков на участке
- `Tc` - среднее время между наладками
- `Ts` - среднее время одной наладки
- λ - интенсивность возникновения заявок на наладку
- μ - интенсивность обслуживания одним наладчиком

Ниже все графики и checkpoint-значения относятся именно к этим числам, а не к абстрактному примеру.

Как этим руководством пользоваться дальше

Если нужно быстро сориентироваться, идите так:

- сначала посмотрите постановку и свои числа;
- потом откройте нужный подпункт;
- на защите начинайте с checkpoint, затем опирайтесь на схему или график и заканчивайте коротким локальным выводом.

Формальный отчёт нужен преподавателю, а это руководство - чтобы быстро восстановить смысл расчётов перед защитой.

Как читать схемы, обозначения, формулы и графики

Сначала всегда смотрим на реальный смысл системы.

Потом смотрим, какие состояния в ней возможны.

После этого вводим обозначения.

Только потом переходим к формулам.

Если на графике что-то меняется по оси `X`, это значит, что мы сознательно меняем именно этот параметр и смотрим, как отвечают метрики по оси `Y`.

Если в графике есть семейство кривых, это значит, что один параметр меняется по оси `X`, а другой остаётся фиксированным и задаёт отдельную кривую.

Главное правило такое: сначала смысл, потом символ, потом формула, потом график, потом простой вывод.

Задача 1. Проектирование колл-центра

Во всех подпунктах задачи 1 мы моделируем поток звонков.

Разница между подпунктами в том, что меняются правила ожидания и отказа.

1.1. Система без очереди

Что происходит в реальности

Клиент звонит в колл-центр.

Если есть свободный оператор, звонок сразу попадает на обслуживание.

Если все операторы заняты, ждать нельзя.

Такой звонок получает отказ.

Что нужно найти в этом подпункте

Нужно понять, как число операторов `n` влияет на три вещи:

- вероятность отказа `P_отк`
- среднее число занятых операторов `M_зан`
- загрузку операторов `K_загр`

Нас особенно интересует первое значение `n`, при котором `P_отк < 0.01`.

Что дано в твоём варианте

В твоём варианте:

- `Tc = 14` с
- `Ts = 65` с
- `λ = 0.0714` 1/с
- `μ = 0.0154` 1/с
- `a = 4.643`

Число `a` показывает, какую среднюю нагрузку создаёт поток звонков.

Если говорить совсем просто, системе в среднем нужно обслуживать нагрузку примерно уровня `4.643` одновременно занятых операторов.

Схема и состояния

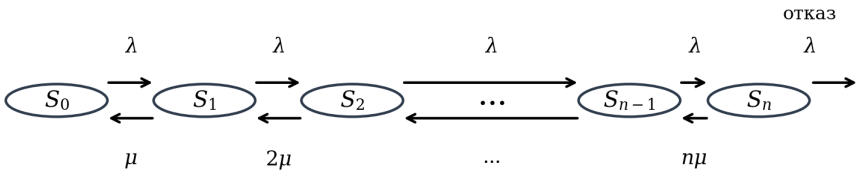


Схема подпункта 1.1.

Состояние `S_k` означает, что занято ровно `k` операторов.

Здесь возможны только состояния от полного покоя до полной занятости.

Правого хвоста очереди нет, потому что ждать нельзя.

На крайнее состояние `S_n` здесь стоит смотреть особенно внимательно.

Именно оно означает, что все `n` операторов заняты.

Обозначения

`n` - число операторов.

`λ` - интенсивность поступления звонков.

`μ` - интенсивность обслуживания одним оператором.

`a = λ / μ` - приведённая нагрузка.

`p_k` - вероятность состояния `S_k`.

`P_отк` - вероятность отказа.

`M_зан` - среднее число занятых операторов.

`K_загр` - доля времени, когда оператор занят в среднем.

Почему формула именно такая

Сначала полезно увидеть перед глазами самые нужные формулы этого подпункта.

$$a = \lambda/\mu$$

Эта величина сразу показывает силу входной нагрузки: сколько “средних занятых операторов” создаёт поток звонков.

$$p_0 = \left(\sum_{k=0}^{n} \frac{a^k}{k!}\right)^{-1}, \qquad p_k = \frac{a^k}{k!} p_0, \quad k = 0, \dots, n$$

Здесь мы распределяем вероятность по всем допустимым состояниям от `0` до `n` занятых операторов.

$$P_{\text{отк}} = p_n, \qquad M_{\text{зан}} = \sum_{k=0}^{n} k\, p_k, \qquad K_{\text{загр}} = M_{\text{зан}}/n$$

Первая формула читает только крайнее состояние полной занятости, а две остальные читают всё распределение состояний целиком.

Мы перебираем все допустимые состояния от `0` до `n` занятых операторов.

У каждого состояния есть свой вес `a^k / k!`.

Нормировка `p_0` нужна затем, чтобы сумма вероятностей всех состояний была равна `1`.

После этого отказ считается очень просто: если новая заявка пришла в состояние `S_n`, обслужить её уже некому.

Поэтому `P_отк = p_n`.

Средняя занятость считается как сумма “сколько операторов занято” умножить на “с какой вероятностью это происходит”.

Загрузка получается делением `M_зан` на `n`.

Здесь важно не перепутать два разных смысла.

`P_отк` берётся только из одного крайнего состояния, потому что отказ возможен только тогда, когда заняты все `n` операторов сразу.

А `M_зан` считается по всем состояниям, потому что средняя занятость должна учитывать каждое возможное число занятых операторов.

Опасный вопрос здесь такой: почему отказ считается по одному состоянию, а занятость - по всему распределению?

Безопасный ответ: потому что отказ связан только с фактом полной занятости, а средняя занятость - это среднее по всем возможным состояниям системы.

Числовой checkpoint

По итоговым расчётным данным для текущего варианта условие `Р_отк < 0.01` впервые выполняется при `n = 11`.

Перед этим при `n = 10` ещё имеем `Р_отк = 0.0125`, `М_зан = 4.585`, `К_загр = 0.459`.

В пороговой точке `n = 11` получаем `Р_отк = 0.00523`, `М_зан = 4.619`, `К_загр = 0.420`.

Это и есть минимально достаточное число операторов по условию задачи.

Ниже в этом руководстве показан один опорный график этого подпункта, а зависимость `М_зан` удобно читать по тем же checkpoint-значениям.

Переход к зависимости занятых операторов

Сначала мы считаем вероятности состояний.

Потом из них получаем `М_зан`.

Поэтому зависимость среднего числа занятых операторов от `n` - это уже не отдельная новая модель, а другое чтение того же распределения состояний.

Как читать зависимость занятых операторов

По оси `X` меняется число операторов `n`.

По оси `Y` показано среднее число занятых операторов `М_зан`.

Параметры `Tc` и `Ts` остаются фиксированными.

Форма кривой означает, что после некоторого момента поток звонков уже не может заметно увеличить фактическую занятость.

Это хорошо видно по переходу от `n = 10` к `n = 11`: `М_зан` растёт только с `4.585` до `4.619`.

Первый безопасный checkpoint здесь - пороговый переход перед выполнением условия по отказу.

На защите лучше сначала сказать, что по оси `Y` почти нет роста, и только потом делать вывод про насыщение кривой.

Безопасный локальный вывод такой: зависимость выходит на полку, а не растёт почти линейно с каждым новым оператором.

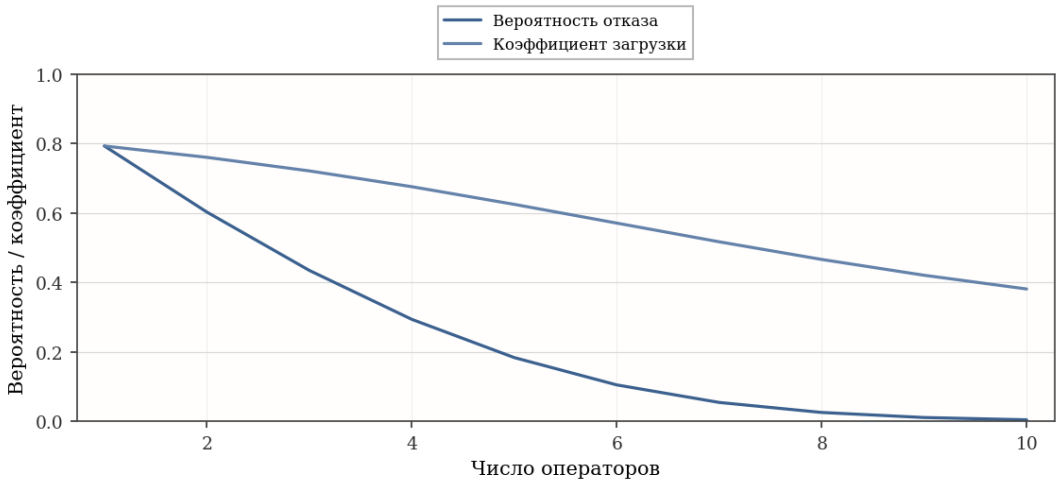
Переход к графику отказа и загрузки

Дальше читаем тот же набор состояний уже с другой стороны.

Нас интересует не среднее число занятых операторов, а крайнее состояние `S_n` и относительная загрузка каждого оператора.

Поэтому на втором графике появляются `Р_отк` и `К_загр`.

Как читать совмещённый график отказа и загрузки



Опорный график 1.1: отказ и загрузка при изменении числа операторов.

По оси `X` снова меняется число операторов `n`.

По оси `Y` показаны две метрики: вероятность отказа `P_отк` и загрузка операторов `K_загр`.

Фиксированными остаются те же `Tc = 14` с и `Ts = 65` с.

Кривая отказа быстро падает, потому что при росте `n` всё реже возникает состояние полной занятости.

Кривая загрузки тоже падает, потому что та же нагрузка распределяется между большим числом операторов.

Локально это видно так: при переходе от `n = 10` к `n = 11` отказ падает с `0.0125` до `0.00523`, а загрузка - с `0.459` до `0.420`.

Первый безопасный checkpoint здесь - та же пороговая пара, потому что на ней сразу видно разницу между потерями и средней работой системы.

На защите безопасно говорить только так: по графику найден минимально достаточный порог по условию отказа, а не “лучшее число операторов вообще”.

Локальный вывод

Для твоего варианта число `n = 11` является первым значением из расчётных данных, где условие по отказу выполнено.

Добавление оператора после `n = 10` почти не увеличивает среднюю занятость, но резко уменьшает вероятность отказа.

Значит, около порога новые операторы нужны не для роста фактической работы, а для снижения потерь.

Это локальный вывод для текущего условия по отказу, а не универсальный оптимум по всем возможным критериям.

Частая опасная путаница здесь такая: переносить сюда логику `1.2` и говорить, что после полной занятости операторов клиент ещё может немного подождать.

Безопасно говорить так: в `1.1` очередь отсутствует совсем, поэтому отказ возникает сразу в состоянии полной занятости, а метрики ожидания из `1.2` сюда переносить нельзя.

Что это значит простыми словами

Поток звонков в твоём варианте по силе похож на работу примерно `4.643` постоянно занятых операторов.

Но если операторов почти столько же, сколько средняя нагрузка, система будет часто упираться в полную занятость.

Поэтому нужен запас.

В этом варианте минимальный безопасный запас по условию задачи достигается при 11 операторах.

Если нужен ответ в одну-две фразы, удобно сказать так: при n = 11 условие P_отк < 0.01 впервые выполняется.

Значит, это минимально достаточный порог по отказу, а дальше операторы дают уже в основном запас, а не резкий рост занятости.

После этого лучше остановиться и не добавлять слова про “лучшее число операторов вообще”, если об этом отдельно не спрашивают.

Безопасная финальная точка такая: для текущего условия по отказу этого ответа уже достаточно.

Что сказать на защите

Ниже четыре короткие карточки.

Карточка 1. Смысл модели

- Что могут спросить: Почему здесь отказ возникает сразу?
- Короткий безопасный ответ: В 1.1 очереди нет. Если все n операторов заняты, новый звонок сразу получает отказ.
- Где чаще всего путаются: Переносят сюда логику из 1.2 и начинают говорить про ожидание.
- Каких слов лучше не говорить: “Клиент здесь немного ждёт”.
- Один практический смысл: Этот подпункт показывает, сколько операторов нужно, если ждать нельзя.

Карточка 2. Смысл метрики

- Что могут спросить: Почему P_отк = p_n?
- Короткий безопасный ответ: Потому что p_n - это вероятность состояния полной занятости. Именно в этом состоянии новый звонок некуда отправить.
- Где чаще всего путаются: Думают, что отказ связан со средней занятостью, а не с крайним состоянием.
- Каких слов лучше не говорить: “Отказ - это когда средняя загрузка большая”.
- Один практический смысл: Мы измеряем, как часто система упирается в полную занятость.

Карточка 3. Как читать график

- Что могут спросить: Как коротко объяснить график отказа и загрузки?
- Короткий безопасный ответ: По оси X растёт число операторов n. По оси Y смотрим P_отк и K_загр. При переходе от n = 10 к n = 11 отказ падает с 0.0125 до 0.00523, а загрузка - с 0.459 до 0.420.
- Где чаще всего путаются: Ожидают, что рост n обязательно сильно поднимет M_зан.
- Каких слов лучше не говорить: “Чем больше операторов, тем они все сильнее заняты”.
- Один практический смысл: Дополнительный оператор здесь даёт запас по отказам, а не большой рост фактической работы.

Карточка 4. Итоговый вывод

- Что могут спросить: Какой итог по твоему варианту?
- Короткий безопасный ответ: Первое n, где P_отк < 0.01, равно 11. Это минимально достаточное значение по условию задачи.
- Где чаще всего путаются: Называют 11 глобально лучшим или “оптимальным” без отдельного критерия.
- Каких слов лучше не говорить: “11 - это оптимальное число операторов”.
- Один практический смысл: Для твоего варианта 11 операторов достаточно, чтобы выполнить лимит по отказу.

1.2. Система с ограниченной очередью

Что происходит в реальности

В этой модели звонок уже не обязательно теряется сразу.

Если все операторы заняты, клиент может подождать.

Но ждать можно только в очереди ограниченной длины m.

Если заняты и операторы, и все места ожидания, тогда происходит отказ.

Что нужно найти в этом подпункте

Нужно посмотреть, как меняются:

- вероятность отказа P_отк
- среднее число занятых операторов M_зан
- загрузка операторов K_загр
- вероятность существования очереди P_оч
- средняя длина очереди L_оч
- занятость мест очереди K_мест

И делать это в двух семействах графиков.

В первом семействе меняется число мест m при фиксированном n.

Во втором семействе меняется число операторов n при фиксированном m.

Что дано в твоём варианте

В твоём варианте для этого подпункта остаются те же:

- Tc = 14 с
- Ts = 65 с
- λ = 0.0714 1/с
- μ = 0.0154 1/с
- a = 4.643

Для опорного объяснения ниже используются характерные точки текущего расчёта:

- семейство по очереди: фиксируем n = 5 и берём m = 1, 5, 15
- семейство по операторам: фиксируем m = 5 и берём n = 5, 8, 11

Схема и состояния

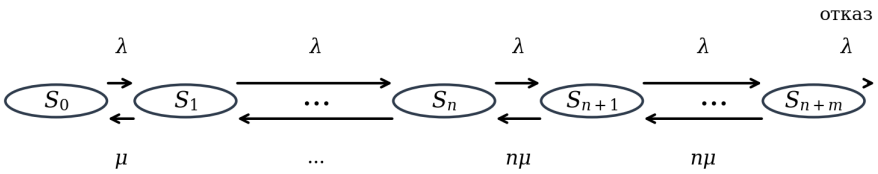


Схема подпункта 1.2.

Состояния обслуживания по-прежнему показывают, сколько операторов занято.

Но теперь справа появляется ещё один хвост состояний.

Этот хвост уже показывает длину очереди от 1 до m.

Это значит, что состояние системы теперь хранит и занятых операторов, и факт ожидания.

Обозначения

n - число операторов.

m - число мест в очереди.

λ и μ - те же интенсивности поступления и обслуживания.

`p_k` - вероятности состояний.

`P_отк` - вероятность отказа.

`M_зан` - среднее число занятых операторов.

`K_загр = M_зан / n` - загрузка операторов.

`P_оч` - вероятность того, что очередь вообще существует.

`L_оч` - средняя длина очереди.

`K_мест = L_оч / m` - какая доля мест ожидания занята в среднем.

Почему формула именно такая

Ниже только тот набор формул, без которого этот подпункт трудно читать самостоятельно.

$$a = \lambda/\mu, \qquad \rho_n = \lambda/(n\mu)$$

Здесь `a` показывает силу потока, а `\rho_n` помогает понять, насколько тяжёлой становится система при выбранном числе операторов `n`.

$$p_0 = \left(\sum_{k=0}^{n} \frac{a^k}{k!} + \frac{a^n}{n!} \sum_{r=1}^{m} \rho_n^r \right)^{-1}$$

Эта нормировка сразу учитывает и состояния обслуживания, и правый хвост состояний ожидания.

$$p_k = \frac{a^k}{k!} p_0, \quad k = 0, \dots, n$$

$$p_{n+r} = \frac{a^n}{n!} \rho_n^r p_0, \quad r = 1, \dots, m$$

Первая строка описывает состояния, где клиенты уже обслуживаются, а вторая — состояния, где после заполнения операторов уже образовалась очередь.

$$P_{\text{отк}} = p_{n+m}$$

Эта формула берёт только последнее состояние цепи, где заняты все операторы и заполнены все места ожидания.

$$M_{\text{зан}} = \sum_{k=0}^{n+m} \min(k, n)\, p_k, \qquad K_{\text{загр}} = M_{\text{зан}}/n$$

Так мы читаем из распределения не потери, а фактическую среднюю занятость операторов.

$$P_{\text{оч}} = \sum_{k=n+1}^{n+m} p_k, \qquad L_{\text{оч}} = \sum_{k=n+1}^{n+m} (k-n)\, p_k, \qquad K_{\text{мест}} = L_{\text{оч}}/m$$

Эти формулы читают уже только правый хвост и поэтому показывают именно цену ожидания.

Теперь вероятность должна распределиться не только по состояниям обслуживания, но и по состояниям ожидания.

Поэтому нормировка становится длиннее.

Она должна покрыть и левую часть схемы, где клиенты обслуживаются, и правую часть, где они ждут.

Отказ снова определяется последним состоянием цепи.

Только теперь это уже не просто `S_n`, а состояние, где заняты все операторы и заполнены все `m` мест ожидания.

Метрики очереди берутся из правого хвоста состояний.

Поэтому этот подпункт и показывает цену снижения отказов: часть нагрузки уходит в ожидание.

Числовой checkpoint

В этом подпункте опорные checkpoint-точки сразу разбиты на два семейства.

Для семейства по очереди фиксируем `n = 5` и дальше сравниваем `m = 1, 5, 15`.

Для семейства по операторам фиксируем `m = 5` и дальше сравниваем `n = 5, 8, 11`.

Ниже в этом руководстве показан один опорный график этого подпункта, а остальные семейства удобно читать по тем же checkpoint-значениям.

Блок `Семейства по числу мест в очереди при фиксированном n`

Здесь по оси `X` меняется число мест в очереди `m`.

По оси `Y` каждый раз смотрим отдельную метрику.

Фиксированным остаётся число операторов `n = 5`.

На защите здесь сначала обязательно проговаривают, что меняется только буфер ожидания, а сам обслуживающий ресурс остаётся тем же.

Поэтому это семейство нельзя читать как эффект добавления новых операторов.

Блок метрики `Отказы`

Смотрим, как меняется `P_отк`, когда мест ожидания становится больше.

Checkpoint-значения такие: при `m = 1` получаем `0.192`, при `m = 5` - `0.0869`, при `m = 15` - `0.0260`.

В этом семействе зависимость убывает, потому что дополнительные места дают клиенту шанс не потеряться сразу.

Смысл формы простой: первые места в очереди помогают сильнее, чем последующие.

Первый безопасный checkpoint здесь - левый край семейства, потому что на нём лучше всего видно выигрыш от первых мест ожидания.

Безопасный локальный вывод: очередь смягчает потери, но по этому семейству зависимостей нельзя говорить, что перегрузка уже устранена полностью.

Блок метрики `Занятые операторы и загрузка`

По оси `X` всё ещё меняется `m`, а по оси `Y` здесь показаны `M_зан` и `K_загр`.

При фиксированном `n = 5` получаем: `M_зан = 3.753, 4.239, 4.522`, а `K_загр = 0.751, 0.848, 0.904` для `m = 1, 5, 15`.

В этом семействе зависимость растёт, потому что очередь удерживает звонки внутри системы и меньше вызовов теряется.

Но рост здесь медленнее, чем падение отказов.

Здесь важно не переоценить рост кривой: операторы не стали работать быстрее, система просто теряет меньше звонков.

Безопасный вывод такой: буфер ожидания удерживает нагрузку внутри системы, а не создаёт новый обслуживающий ресурс.

Блок метрики `Очередь`

Здесь по оси `X` снова стоит `m`, а по оси `Y` уже смотрим `P_оч`, `L_оч` и `K_мест`.

При `n = 5` имеем: `P_оч = 0.192, 0.507, 0.690`; `L_оч = 0.192, 1.445, 4.585`; `K_мест = 0.192, 0.289, 0.306` для `m = 1, 5, 15`.

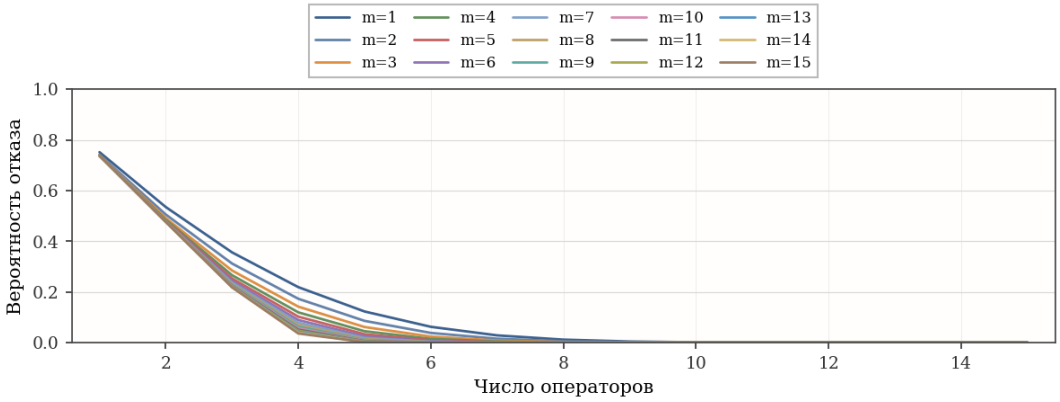
Здесь зависимости растут, потому что уменьшение потерь переводит часть клиентов в ожидание.

По-человечески это читается так: очередь спасает звонки от мгновенного отказа, но делает систему более “ожидающей”.

Первый безопасный вывод здесь локальный: снижение отказов оплачивается ростом ожидания.

Опасно по одному этому семейству говорить, что большая очередь сама по себе лучше добавления операторов.

Блок `Семейства по числу операторов при фиксированном m`



Опорный график 1.2: отказ при изменении числа операторов для разных очередей.

Во втором семействе по оси `X` уже меняется число операторов `n`.

По оси `Y` снова смотрим те же метрики.

Фиксированным остаётся число мест ожидания `m = 5`.

Здесь, наоборот, нужно сразу проговорить, что меняется именно обслуживающий ресурс, а буфер ожидания остаётся тем же.

Поэтому это семейство показывает уже не эффект ожидания, а эффект добавления операторов.

Блок метрики `Отказы`

Смотрим, как меняется `P_отк`, когда добавляем операторов.

Checkpoint-значения такие: при `n = 5` получаем `0.0869`, при `n = 8` - `0.00333`, при `n = 11` - `0.000070`.

Во втором семействе зависимость падает быстрее, чем в предыдущем.

Причина здесь прямая: теперь мы добавляем не буфер ожидания, а сам обслуживающий ресурс.

Первый безопасный checkpoint здесь - сравнение левого края семейства с более свободным режимом при том же буфере ожидания.

Безопасный локальный вывод: дополнительные операторы убирают причину отказа быстрее, чем одни места ожидания.

Блок метрики `Занятые операторы и загрузка`

По оси `X` меняется `n`, по оси `Y` показаны `M_зан` и `K_загр`, а `m = 5` остаётся фиксированным.

Получаем: `M_зан = 4.239, 4.627, 4.643`, а `K_загр = 0.848, 0.578, 0.422` для `n = 5, 8, 11`.

Среднее число занятых операторов растёт совсем немного.

Загрузка каждого оператора падает сильно, потому что появляется резерв.

Если M_зан растёт слабо, а K_загр падает сильно, это не противоречие.

Это значит, что фактическая работа почти насыщена, а новый оператор добавляет в основном резерв.

Блок метрики Очередь

По оси X меняется n, а по оси Y здесь показаны P_оч, L_оч и K_мест.

При m = 5 имеем: P_оч = 0.507, 0.0654, 0.00375; L_оч = 1.445, 0.133, 0.00624; K_мест = 0.289, 0.0265, 0.00125 для n = 5, 8, 11.

Во втором семействе зависимости быстро убывают, потому что дополнительные операторы убирают саму причину ожидания.

В простом чтении это означает: если людей на линии хватает, очередь почти перестаёт быть нужна.

На защите безопасно говорить, что это семейство зависимостей показывает уменьшение самой необходимости ждать.

Но нельзя по одному этому семейству объявлять какую-то пару (n, m) универсально лучшей.

Локальный вывод

Для твоего варианта расширение очереди уменьшает потери, но одновременно переносит часть нагрузки в ожидание.

Добавление операторов действует сильнее.

Оно одновременно уменьшает и отказы, и длину очереди.

Значит, очередь - это способ смягчить перегрузку, а дополнительные операторы - способ убрать её причину.

Это сравнение двух семейств, а не готовое объявление единственного лучшего решения без отдельного критерия.

Частая опасная путаница здесь такая: смешивать 1.2 с 1.1 или 1.3, как будто меняется только длина рисунка, а сама модель остаётся той же.

Безопасно говорить так: по сравнению с 1.1 здесь появляется конечный буфер ожидания, а по сравнению с 1.3 этот буфер всё ещё ограничен, поэтому именно в 1.2 можно одновременно обсуждать и очередь, и отказ из-за заполнения всех мест.

Что это значит простыми словами

Если системе не хватает операторов, одна только очередь не решает проблему до конца.

Она даёт клиенту место подождать, но не создаёт новый канал обслуживания.

Поэтому в твоём варианте добавление операторов снимает напряжение быстрее, чем простое увеличение m.

Если нужен ответ в одну-две фразы, удобно сказать так: при m = 5 переход от n = 5 к n = 8 снижает P_отк с 0.0869 до 0.00333 и L_оч с 1.445 до 0.133.

Значит, в твоём варианте операторы снимают перегрузку сильнее, чем одна только очередь.

После этого лучше остановиться и не добавлять, что теперь достаточно просто всегда увеличивать либо только m, либо только n.

Безопасная финальная точка такая: для твоего варианта этого локального сравнения уже достаточно, если дальше не просят отдельный критерий выбора.

Что сказать на защите

Ниже четыре короткие карточки.

Карточка 1. Смысл модели

- Что могут спросить: Когда здесь происходит отказ?
- Короткий безопасный ответ: Отказ здесь возникает только тогда, когда заняты все `n` операторов и заполнены все `m` мест ожидания. Пока место в очереди есть, звонок не теряется сразу.
- Где чаще всего путаются: Забывают, что очередь здесь ограничена.
- Каких слов лучше не говорить: “Очередь здесь бесконечная”.
- Один практический смысл: Это модель обмена между потерями и ожиданием.

Карточка 2. Смысл метрики

- Что могут спросить: Что даёт увеличение `m` при `n = 5`?
- Короткий безопасный ответ: Потери падают с `0.192` при `m = 1` до `0.0260` при `m = 15`, но очередь растёт: `L_оч` растёт с `0.192` до `4.585`.
- Где чаще всего путаются: Говорят только про пользу очереди и забывают цену в виде ожидания.
- Каких слов лучше не говорить: “Очередь убирает отказы без побочного эффекта”.
- Один практический смысл: Дополнительные места переводят часть потерь в ожидание.

Карточка 3. Как читать график

- Что могут спросить: Как коротко объяснить разницу двух семейств графиков?
- Короткий безопасный ответ: В первом семействе меняется `m` при фиксированном `n = 5`. Во втором меняется `n` при фиксированном `m = 5`. Во втором случае отказы и очередь падают быстрее, потому что добавляется сам обслуживающий ресурс.
- Где чаще всего путаются: Переносят числа из одного семейства в другое, как будто фиксированные параметры не важны.
- Каких слов лучше не говорить: “Это одни и те же графики, просто нарисованы по-разному”.
- Один практический смысл: Нужно отдельно различать “добавили буфер” и “добавили операторов”.

Карточка 4. Итоговый вывод

- Что могут спросить: Что сильнее помогает в твоём варианте - очередь или операторы?
- Короткий безопасный ответ: Операторы. При `m = 5` переход от `n = 5` к `n = 8` снижает `P_отк` с `0.0869` до `0.00333` и `L_оч` с `1.445` до `0.133`.
- Где чаще всего путаются: Делают общий вывод, что достаточно просто увеличивать `m`.
- Каких слов лучше не говорить: “Главное просто добавить места в очереди”.
- Один практический смысл: Очередь смягчает перегрузку, а операторы убирают её причину.

1.3. Система с неограниченной очередью

Что происходит в реальности

Теперь клиент может ждать сколько угодно долго.

Формально длина очереди не ограничена сверху.

Но у такой схемы есть своя цена.

Если система слишком перегружена, устойчивый режим вообще может не существовать.

Что нужно найти в этом подпункте

Нужно посмотреть:

- среднее число занятых операторов `M_зан`
- загрузку операторов `K_загр`
- вероятность существования очереди `P_оч`
- среднюю длину очереди `L_оч`

Перед этим обязательно нужно проверить стационарность.

Что дано в твоём варианте

Для твоего варианта снова имеем:

- `Tc = 14` с

- `Ts = 65` с
- `λ = 0.0714` 1/с
- `μ = 0.0154` 1/с
- `a = 4.643`

Первая стационарная точка по итоговым расчётным данным возникает при `n = 5`.

Точки `n = 1, 2, 3, 4` нестационарны и не должны получать выдуманные стационарные метрики.

Схема и состояния

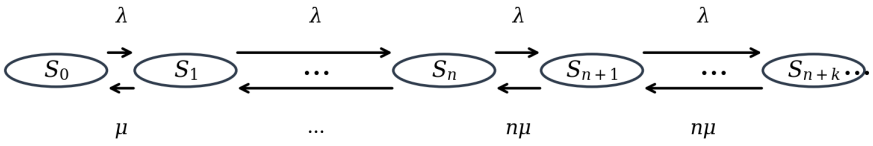


Схема подпункта 1.3.

Слева остаются состояния обслуживания.

Справа теперь бесконечный хвост очереди.

Это значит, что после заполнения всех операторов очередь может продолжаться дальше без верхней границы.

Именно поэтому здесь надо отдельно проверять, не “разбегается” ли хвост вправо.

Обозначения

`ρ_n = λ / (n μ)` - коэффициент нагрузки при фиксированном `n`.

`P_wait` - вероятность того, что новая заявка застанет всех `n` операторов занятыми.

`P_оч` - вероятность существования очереди в стационарном режиме.

`L_оч` - средняя длина очереди.

`M_зан` и `K_загр` - среднее число занятых операторов и их загрузка.

Почему формула именно такая

Здесь тоже полезно сначала увидеть минимум формул, а уже потом читать словесное объяснение.

$$a = \lambda/\mu, \qquad \rho_n = \lambda/(n\mu)$$

`a` показывает силу нагрузки, а `\rho_n` сразу отвечает на главный вопрос этого подпункта: существует ли стационарный режим при выбранном `n`.

$$p_0 = \left(\sum_{k=0}^{n-1} \frac{a^k}{k!} + \frac{a^n}{n!(1-\rho_n)}\right)^{-1}, \qquad \rho_n < 1$$

Эта формула допустима только в стационарной зоне, поэтому она сразу отделяет рабочие точки от нестационарных.

$$P_{\text{wait}} = \frac{a^n}{n!(1-\rho_n)} p_0$$

Это вероятность того, что новая заявка застанет всех операторов занятыми и попадёт в режим ожидания.

$$M_{\text{зан}} = a, \qquad K_{\text{загр}} = a/n$$

Здесь средняя занятость в устойчивом режиме задаётся самой нагрузкой, а коэффициент загрузки показывает, как эта нагрузка делится между `n` операторами.

$$P_{\text{оч}} = P_{\text{wait}}\rho_n, \qquad L_{\text{оч}} = \frac{P_{\text{wait}}\rho_n}{1-\rho_n}$$

Эти формулы уже читают не просто факт ожидания новой заявки, а тяжесть самой очереди в стационарном режиме.

Бесконечную очередь нельзя считать без условия устойчивости.

Если `ρ_n >= 1`, поток требований не успевает разгружаться и стационарное распределение не существует.

Поэтому первым делом проверяем условие `ρ_n < 1`.

Только после этого можно считать `p_0`, `P_wait`, `P_оч` и `L_оч`.

Для твоего варианта условие `ρ_n < 1` впервые выполняется при `n = 5`.

Логика этого шага простая: сначала мы отвечаем не на вопрос “какая очередь”, а на вопрос “имеют ли стационарные средние вообще право здесь существовать”.

Именно из этого условия и появляется граница, после которой уже можно честно читать метрики из стационарного распределения.

Опасный вопрос здесь такой: почему нельзя просто посчитать те же метрики и в неустойчивом режиме?

Безопасный ответ: потому что в неустойчивом режиме очередь в среднем не стабилизируется, и стационарные `P_wait`, `P_оч` и `L_оч` там не имеют честного фиксированного значения.

Числовой checkpoint

Первая допустимая стационарная точка - `n = 5`.

В ней `ρ_n = 0.929`, `P_wait = 0.828`, `M_зан = 4.643`, `K_загр = 0.929`, `P_оч = 0.768`, `L_оч = 10.758`.

Дальше для сравнения берём ещё `n = 7` и `n = 10`.

При `n = 7` получаем `ρ_n = 0.663`, `P_wait = 0.245`, `P_оч = 0.163`, `L_оч = 0.483`.

При `n = 10` получаем `ρ_n = 0.464`, `P_wait = 0.0230`, `P_оч = 0.0107`, `L_оч = 0.0199`.

Ниже в этом руководстве показан один опорный график этого подпункта, а остальные зависимости удобно читать по тем же checkpoint-значениям.

Блок `Вероятность ожидания`

По оси `X` меняется число операторов `n`.

По оси `Y` в этом блоке смотрим вероятность ожидания `P_wait`.

Фиксированными остаются `Tc` и `Ts`.

Соответствующая зависимость резко убывает после первой стационарной точки.

Это значит, что даже небольшой отход от границы устойчивости сильно уменьшает шанс того, что новый клиент попадёт в режим “все заняты”.

Важно не путать `P_wait` и `P_оч`.

`P_wait` относится к новой входящей заявке, а `P_оч` - к стационарному факту наличия очереди.

Первый безопасный checkpoint здесь - первая стационарная точка, потому что именно от неё эту зависимость уже можно читать как честную стационарную кривую.

Безопасный локальный вывод: кривая убывает из-за удаления от границы устойчивости, а не просто “потому что операторов стало больше”.

Блок `Занятые операторы и загрузка`

По оси `X` меняется `n`, а по оси `Y` показаны `M_зан` и `K_загр`.

В стационарной зоне `M_зан` остаётся равным `4.643` на всех выбранных опорных точках.

Зато `K_загр` падает с `0.929` при `n = 5` до `0.663` при `n = 7` и до `0.464` при `n = 10`.

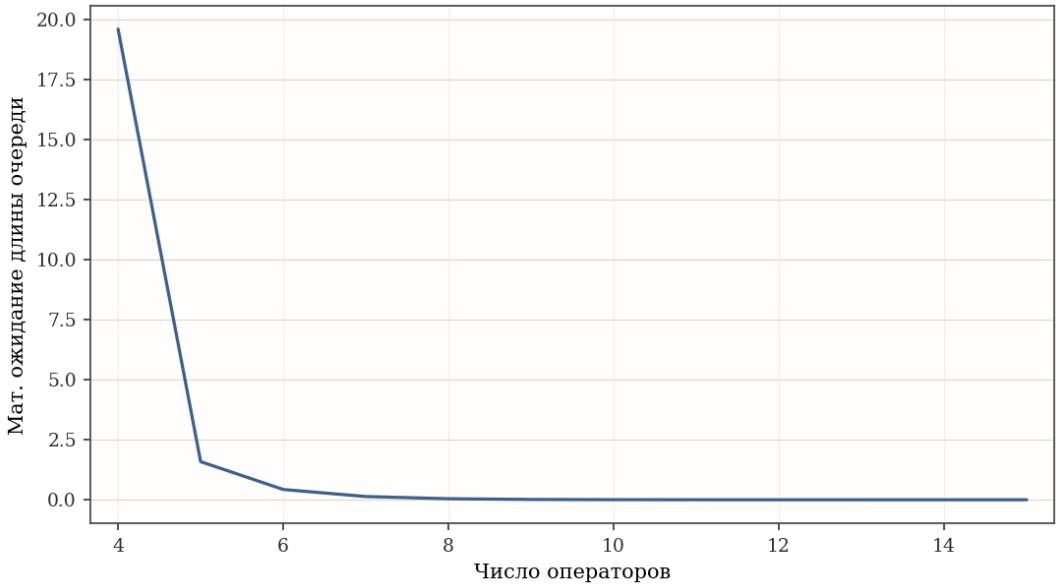
Форма соответствующей зависимости означает простую вещь: сама обслуженная нагрузка не растёт, растёт только запас свободной мощности.

Первый безопасный checkpoint здесь - сравнение первой рабочей точки с более свободным режимом.

По почти горизонтальной кривой `M_зан` нельзя говорить, что добавление операторов бесполезно.

Безопасный вывод: меняется не обслуженная нагрузка, а запас мощности системы.

Блок `Очередь`



Опорный график 1.3: средняя длина очереди при увеличении числа операторов.

По оси `X` меняется `n`, а по оси `Y` здесь стоят `P_оч` и `L_оч`.

При `n = 5` очередь тяжёлая: `P_оч = 0.768`, `L_оч = 10.758`.

При `n = 7` она уже намного легче: `0.163` и `0.483`.

При `n = 10` эти значения становятся `0.0107` и `0.0199`.

Форма графиков показывает, что бесконечная очередь особенно опасна рядом с границей устойчивости.

Здесь безопаснее первым назвать самую тяжёлую стационарную точку, а не сразу перескакивать к дальнему правому краю графика.

Безопасный локальный вывод здесь такой: около границы устойчивости очередь тяжёлая, а дальше быстро облегчается.

Локальный вывод

В твоём варианте нельзя начинать объяснение этого подпункта с опорного графика.

Сначала нужно проверить, существует ли стационарный режим вообще.

Он появляется только с `n = 5`.

После этого уже видно, что каждое добавление операторов быстро облегчает очередь.

Частая опасная путаница здесь такая: думать, что `1.3` - это просто `1.2` с очень большим `m`, или что `1.4` отличается только более лёгкими числами.

Безопасно говорить так: по сравнению с `1.2` здесь нет конечной границы очереди и поэтому сначала обязателен разговор про стационарность, а по сравнению с `1.4` здесь хвост разгружается только обслуживанием, без ухода клиентов.

Что это значит простыми словами

Бесконечная очередь не значит “можно поставить мало операторов и просто ждать”.

Если операторов слишком мало, система не успевает разгружаться.

Для твоего варианта первая рабочая стационарная точка - это `5` операторов.

Но около неё очередь ещё очень тяжёлая, поэтому нужен дополнительный запас.

Если нужен ответ в одну-две фразы, удобно сказать так: при `n = 5` система впервые стационарна, но `L_оч = 10.758` ещё слишком велика.

Значит, одного выполнения условия стационарности мало: после порога нужен ещё запас по очереди.

После этого лучше остановиться и не добавлять стационарные числа для точек до границы устойчивости.

Безопасная финальная точка такая: я назвал первую рабочую точку и сказал, что после неё ещё нужен запас по очереди.

Что сказать на защите

Ниже четыре короткие карточки.

Карточка 1. Смысл модели

- Что могут спросить: Что здесь главное в модели?
- Короткий безопасный ответ: Очередь здесь бесконечная, поэтому сначала нужно проверить, существует ли стационарный режим. Без этого средние метрики нельзя честно считать.
- Где чаще всего путаются: Сразу переходят к графикам и забывают про условие устойчивости.
- Каких слов лучше не говорить: “Раз очередь бесконечная, считать можно всегда”.
- Один практический смысл: Первый вопрос здесь не “какая очередь”, а “устойчива ли система”.

Карточка 2. Смысл метрики

- Что могут спросить: Почему в твоём варианте сначала смотрим `ρ_n < 1`?
- Короткий безопасный ответ: Потому что это условие стационарности. Для твоего варианта оно впервые выполняется только при `n = 5`.
- Где чаще всего путаются: Пытаются приписать стационарные числа точкам `n = 1, 2, 3, 4`.
- Каких слов лучше не говорить: “При `n = 4` очередь просто очень большая”.
- Один практический смысл: До `n = 5` у нас нет честных стационарных средних.

Карточка 3. Как читать график

- Что могут спросить: Чем отличаются `P_wait` и `P_оч`?
- Короткий безопасный ответ: `P_wait` относится к новой заявке и показывает, увидит ли она всех операторов занятыми. `P_оч` относится к текущему состоянию системы. При `n = 5` это `0.828` и `0.768`, и это не одно и то же.
- Где чаще всего путаются: Называют обе величины одной и той же вероятностью очереди.
- Каких слов лучше не говорить: “Это одна и та же вероятность”.
- Один практический смысл: Одна метрика про новую заявку, другая про текущее состояние системы.

Карточка 4. Итоговый вывод

- Что могут спросить: Какой главный вывод по твоему варианту?
- Короткий безопасный ответ: Первая рабочая точка - `n = 5`, но около неё очередь ещё тяжёлая: `L_оч = 10.758`. При `n = 7` и `n = 10` очередь становится намного легче.
- Где чаще всего путаются: Думают, что если условие стационарности уже выполнено, то этого автоматически достаточно.
- Каких слов лучше не говорить: “Раз `n = 5` стационарно, значит проблема уже решена”.
- Один практический смысл: Выполнить условие стационарности мало, нужен ещё запас по очереди.

1.4. Система с неограниченной очередью и уходом клиентов

Что происходит в реальности

Теперь клиент может не дождаться обслуживания и уйти из очереди сам.

Значит, очередь формально бесконечна, но её хвост уже не бесконтрольно накапливается.

Часть клиентов покидает систему раньше.

Из-за этого сама модель устроена уже немного иначе.

Что нужно найти в этом подпункте

Нужно посмотреть:

- среднее число занятых операторов `M_зан`
- загрузку операторов `K_загр`
- вероятность существования очереди `P_оч`
- среднюю длину очереди `L_оч`

И отдельно нужно понять, почему здесь вообще можно считать бесконечный хвост численно.

Что дано в твоём варианте

В твоём варианте:

- `Tc = 14` с
- `Ts = 65` с
- `Tw = 106` с
- λ `= 0.0714` 1/с
- μ `= 0.0154` 1/с
- ν `= 0.00943` 1/с
- `a = 4.643`

Для контролируемого усечения используются:

- `epsilon_probability = 1e-12`
- `epsilon_queue = 1e-12`
- `max_state = 100000`

Максимальный оставшийся хвост по всему sweep не превышает `4.791e-14` по вероятности и `8.497e-13` по вкладу в среднюю длину очереди.

Схема и состояния

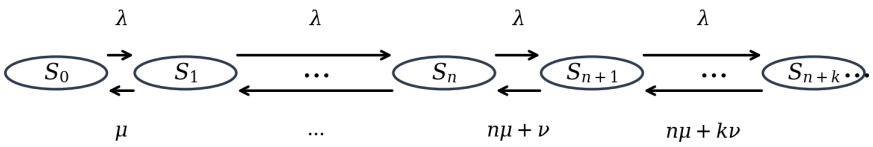


Схема подпункта 1.4.

Состояние `S_k` означает общее число заявок в системе.

Если `k <= n`, все эти заявки либо обслуживаются, либо часть операторов свободна.

Если `k > n`, то `n` заявок обслуживаются, а остальные `k - n` стоят в очереди.

Теперь у правого хвоста есть ещё один механизм разгрузки.

Это уход клиентов из очереди.

Обозначения

`ν = 1 / Tw` - интенсивность ухода из очереди.

`β_k` - интенсивность перехода вправо, то есть поступления новой заявки.

`δ_k` - суммарная интенсивность перехода влево.

`p_k` - стационарная вероятность состояния после численного построения.

`P_оч`, `L_оч`, `M_зан`, `K_загр` - те же метрики, что и раньше.

Почему формула именно такая

В этом подпункте достаточно держать перед глазами четыре опорные формулы.

$$\nu = 1/T_w, \qquad \beta_k = \lambda$$

Скорость прихода новых заявок не меняется, а `\nu` задаёт, как быстро клиенты могут уходить из очереди.

$$\delta_k = \min(k, n)\mu + \max(k - n, 0)\nu$$

Здесь в одной строке собраны оба механизма разгрузки: обслуживание до границы `n` и уход клиентов после появления хвоста.

$$p_k = p_{k-1}\beta_k/\delta_k, \quad k \geq 1, \qquad p_0 = \left(1 + \sum_{k=1}^{\infty}\prod_{i=1}^{k}\beta_i/\delta_i\right)^{-1}$$

Так вероятности состояний строятся шаг за шагом, а затем вся цепь закрывается нормировкой.

$$M_{\text{зан}} = \sum_{k=0}^{\infty}\min(k, n)\, p_k, \qquad K_{\text{загр}} = M_{\text{зан}}/n$$

Эти формулы отвечают за занятость операторов и не смотрят отдельно на смысл хвоста, а просто читают всё распределение целиком.

$$P_{\text{оч}} = \sum_{k=n+1}^{\infty} p_k, \qquad L_{\text{оч}} = \sum_{k=n+1}^{\infty} (k-n)\, p_k$$

А здесь, наоборот, читается только хвост состояний, где после заполнения операторов уже существует очередь.

В этой модели приход новых заявок не меняется, поэтому `β_k = λ`.

А вот обратный переход зависит от того, заполнены ли уже операторы.

Пока `k <= n`, система разгружается только обслуживанием.

Когда `k > n`, к обслуживанию добавляется ещё и уход клиентов.

Именно поэтому `δ_k = min(k, n) μ + max(k - n, 0) ν`.

Дальше вероятности `p_k` строятся рекуррентно.

Хвост бесконечный, поэтому суммирование идёт не до “какого-то красивого числа”, а до контролируемого порога точности.

Здесь формула для `δ_k` появляется прямо из чтения схемы состояний.

До границы `n` систему разгружает только обслуживание, а после границы хвост уменьшается уже двумя путями: обслуживанием и уходом клиентов из очереди.

Опасный вопрос здесь такой: почему в хвосте можно складывать обслуживание и уход?

Безопасный ответ: потому что это два разных независимых способа выйти из состояния `k > n`, и суммарная интенсивность выхода складывается из обеих частей.

Блок `Рекуррентное построение и усечение хвоста`

Сначала строится последовательность `p_k`.

Потом хвост обрывается только тогда, когда его остаток становится меньше заданных `1e-12`.

Это не случайная отсечка.

Это контролируемое численное приближение с явной оценкой остатка.

Поэтому метрики очереди здесь берутся не “из обрезанного куска на глаз”, а из хвоста, для которого заранее известна верхняя граница недоучтённого остатка.

Опасный вопрос здесь такой: почему усечение не искажает вывод?

Безопасный ответ: потому что остаток хвоста не объявлен нулём, а честно ограничен сверху, и этот предел очень мал для текущей учебной точности.

Числовой checkpoint

По всему диапазону операторов максимальный недоучтённый хвост ограничен значениями `4.791e-14` по вероятности и `8.497e-13` по среднему числу заявок в очереди.

Значит, в пределах текущей учебной точности этот остаток пренебрежимо мал.

Но важно говорить честно: он не равен нулю, а только очень мал и явно ограничен сверху.

Ниже в этом руководстве показан один опорный график этого подпункта, а остальные зависимости удобно читать по тем же checkpoint-значениям.

Блок `Занятые операторы и загрузка`

По оси `X` меняется число операторов `n`.

По оси `Y` показаны `M_зан` и `K_загр`.

Фиксированными остаются `Tc`, `Ts` и `Tw`.

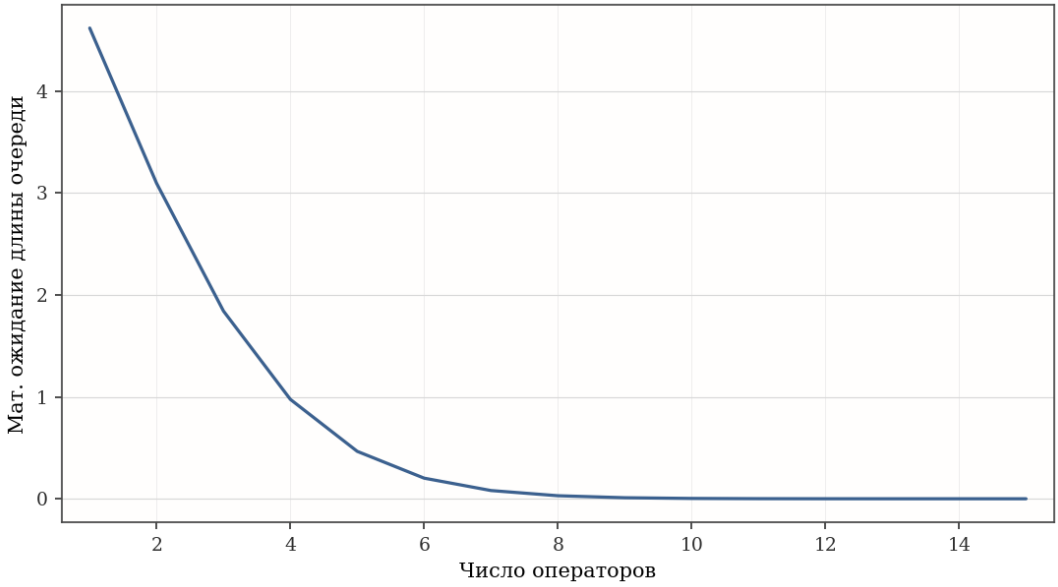
На характерных точках имеем: при `n = 1` получаем `M_зан = 0.998`, `K_загр = 0.998`; при `n = 5` - `4.048` и `0.810`; при `n = 8` - `4.583` и `0.573`.

Форма соответствующей зависимости означает, что среднее число занятых операторов выходит к уровню нагрузки, а относительная загрузка каждого оператора падает по мере появления резерва.

Первый безопасный checkpoint здесь - переход от малого числа операторов к режиму, где `M_зан` уже почти вышло на свой уровень.

Безопасный вывод: зависимость `M_зан` насыщается, а зависимость `K_загр` показывает цену резерва.

Блок `Очередь и уход клиентов`



Опорный график 1.4: средняя длина очереди при увеличении числа операторов.

По оси `X` меняется `n`.

По оси `Y` здесь показаны `P_оч` и `L_оч`.

При `n = 1` имеем `P_оч = 0.988`, `L_оч = 5.944`.

При `n = 5` эти значения снижаются до `0.382` и `0.971`.

При `n = 8` они становятся `0.0530` и `0.0979`.

График убывает, потому что часть хвоста теперь разгружается не только обслуживанием, но и уходом клиентов.

Но это не значит, что очередь исчезает автоматически на малых `n`.

На защите сначала лучше назвать, что в этом графике работают сразу два механизма разгрузки хвоста.

Безопасный локальный вывод: уход клиентов ускоряет спад очереди, но не делает малое `n` автоматически приемлемым.

Локальный вывод

Уход клиентов делает хвост очереди заметно легче, чем в подпункте `1.3`.

Но он не заменяет операторов.

Если операторов мало, очередь всё равно остаётся длинной.

Зато при умеренном увеличении `n` хвост уменьшается очень быстро.

Частая опасная путаница здесь такая: переносить выводы из `1.3` один в один и забывать, что здесь у хвоста появился второй канал разгрузки.

Безопасно говорить так: `1.4` нельзя объяснять как ту же бесконечную очередь, потому что здесь после границы обслуживания хвост уменьшается не только из-за работы операторов, но и из-за ухода клиентов.

Что это значит простыми словами

Этот подпункт описывает не “идеальную” очередь, а очередь, из которой люди могут уйти.

Из-за этого система легче разгружается.

Но малая численность операторов всё равно остаётся проблемой.

Теперь часть перегрузки снимается не только обслуживанием, но и потерей терпения клиентов.

Если нужен ответ в одну-две фразы, удобно сказать так: при `n = 5` уже имеем `P_оч = 0.382` и `L_оч = 0.971`, а при `n = 8` они снижаются до `0.0530` и `0.0979`.

Значит, уход клиентов заметно облегчает хвост, но не заменяет достаточное число операторов.

После этого лучше остановиться и не добавлять, что уход клиентов сам по себе решает проблему перегрузки.

Безопасная финальная точка такая: хвост облегчается, но достаточное число операторов всё равно нужно.

Что сказать на защите

Ниже четыре короткие карточки.

Карточка 1. Смысл модели

- Что могут спросить: Что меняет уход клиентов в этой модели?
- Короткий безопасный ответ: Он добавляет ещё один способ разгрузки хвоста очереди. Клиенты могут уйти, если ждут слишком долго.
- Где чаще всего путаются: Думают, что уход клиентов полностью убирает очередь.
- Каких слов лучше не говорить: “С уходом очереди уже нет”.
- Один практический смысл: Хвост становится легче, но малое число операторов всё равно опасно.

Карточка 2. Смысл метрики

- Что могут спросить: Почему здесь можно считать весь просмотренный диапазон?
- Короткий безопасный ответ: Потому что хвост считается с контролируемым усечением. Остаток ограничен сверху: `4.791e-14` по вероятности и `8.497e-13` по средней длине очереди.
- Где чаще всего путаются: Говорят, что ошибка здесь равна нулю.
- Каких слов лучше не говорить: “Мы просто обрезали хвост и ошибки нет”.
- Один практический смысл: Это честное численное приближение, а не случайная отсечка.

Карточка 3. Как читать график

- Что могут спросить: Как коротко объяснить график очереди в этом подпункте?
- Короткий безопасный ответ: По оси `X` меняется `n`, по оси `Y` смотрим `P_оч` и `L_оч`. При `n = 1` это `0.988` и `5.944`, при `n = 5` уже `0.382` и `0.971`, при `n = 8` - `0.0530` и `0.0979`.
- Где чаще всего путаются: Сравнивают этот подпункт с `1.3` и забывают, что здесь есть `Tw` и уход клиентов.
- Каких слов лучше не говорить: “Это тот же самый график, только с другим рисунком”.
- Один практический смысл: Уход клиентов заметно режет хвост очереди.

Карточка 4. Итоговый вывод

- Что могут спросить: Какой безопасный вывод по твоему варианту?
- Короткий безопасный ответ: Уход клиентов помогает, но не заменяет операторов. При малом `n` очередь всё ещё длинная, а при росте `n` хвост быстро уменьшается.
- Где чаще всего путаются: Выдают уход клиентов за полное решение перегрузки.
- Каких слов лучше не говорить: “Можно не добавлять операторов, люди сами уйдут”.
- Один практический смысл: Уход клиентов снижает перегрузку, но запас обслуживания всё равно нужен.

Задача 2. Производственный участок

Теперь вместо звонков рассматриваем станки и наладчиков.

Логика похожая: есть поток требований на обслуживание и есть ограниченный ресурс, который эти требования обрабатывает.

Но смысл здесь уже другой.

Здесь мы считаем не клиентов и операторов, а неисправные станки и наладчиков.

2.1. Метрики производственного участка по числу наладчиков

Что происходит в реальности

На участке работает `N = 36` станков.

Станок может сломаться.

После поломки он ждёт наладчика или сразу попадает на ремонт.

Если наладчиков мало, часть станков будет простаивать в ожидании обслуживания.

Что нужно найти в этом подпункте

Нужно посмотреть, как число наладчиков `r` влияет на:

- среднее число простаивающих станков `M_пр`
- среднее число ожидающих станков `M_ож`
- вероятность ожидания обслуживания `P_ож`
- среднее число занятых наладчиков `M_зан`
- загрузку наладчиков `K_загр`

Что дано в твоём варианте

В твоём варианте:

- `N = 36`
- `Tc = 104` мин
- `Ts = 50` мин
- `λ = 0.00962` 1/мин
- `μ = 0.0200` 1/мин

Для опорного объяснения используются точки `r = 1, 10, 20, 33`.

Это опорные точки, выбранные для короткого разбора по текущим расчётным данным.

Схема и состояния

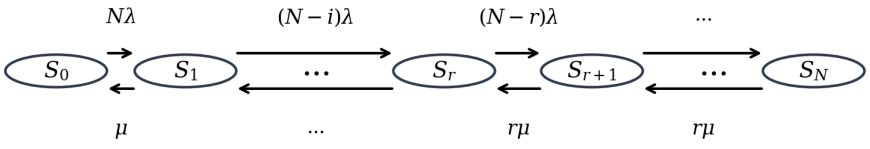


Схема подпункта 2.1.

Состояние `S_i` означает, что неисправны ровно `i` станков.

Если `i <= r`, всех неисправных станков можно обслуживать без ожидания.

Если `i > r`, часть станков уже ждёт своей очереди к наладчику.

Значит, здесь очередь появляется не у клиентов, а у сломанных станков.

Обозначения

`r` - число наладчиков.

`λ_i = (N - i) λ` - интенсивность очередной поломки.

`μ_i = min(i, r) μ` - интенсивность восстановления.

`p_i` - вероятность состояния `S_i`.

`M_пр` - среднее число простаивающих станков.

`M_ож` - среднее число станков, которые уже ждут наладчика.

`P_ож` - вероятность того, что новый отказавший станок попадёт в ожидание.

`M_зан` и `K_загр` - среднее число занятых наладчиков и их загрузка.

Почему формула именно такая

Здесь нужны только формулы, которые связывают число неисправных станков с тем, как из состояний потом читаются все метрики.

$$\lambda_i = (N - i)\lambda, \qquad \mu_i = \min(i, r)\mu$$

Первая формула смотрит на число ещё исправных станков, а вторая — на то, сколько ремонтов реально можно вести одновременно.

$$p_i = p_{i-1}\lambda_{i-1}/\mu_i, \quad i = 1, \dots, N, \qquad p_0 = \left(1 + \sum_{i=1}^{N} \prod_{j=1}^{i} \lambda_{j-1}/\mu_j\right)^{-1}$$

Так шаг за шагом строится всё стационарное распределение по числу неисправных станков.

$$M_{\text{пр}} = \sum_{i=0}^{N} i\, p_i, \qquad M_{\text{ож}} = \sum_{i=0}^{N} \max(i - r, 0)\, p_i$$

Здесь первая формула считает все простаивающие станки, а вторая выделяет только ту часть, которая уже ждёт наладчика.

$$P_{\text{ож}} = \frac{\sum_{i=r}^{N-1} (N - i) p_i}{\sum_{i=0}^{N-1} (N - i) p_i}$$

Это именно вероятность того, что новый отказ попадёт в ожидание, а не просто календарный факт наличия очереди в системе.

$$M_{\text{зан}} = \sum_{i=0}^{N} \min(i, r)\, p_i, \qquad K_{\text{загр}} = M_{\text{зан}}/r$$

Эти две формулы уже читают тот же набор состояний с точки зрения работы самих наладчиков.

Если исправных станков много, новых поломок может быть больше.

Поэтому интенсивность отказа зависит от `N - i`.

С другой стороны, ремонтировать одновременно можно не больше, чем есть наладчиков.

Поэтому интенсивность восстановления ограничивается min(i, r).

После этого стационарные вероятности p_i удобно строить рекуррентно.

Из одного и того же распределения потом считаются и простои, и ожидание, и занятость наладчиков.

Здесь важно помнить, что состояние S_i считает именно неисправные станки, а не занятых наладчиков.

Поэтому λ_i смотрит на число ещё исправных станков N - i, а μ_i - на то, сколько ремонтов реально можно вести одновременно, то есть не больше r.

Опасный вопрос здесь такой: почему и простои, и ожидание, и занятость читаются из одного p_i?

Безопасный ответ: потому что p_i описывает полную вероятность каждого состояния по числу неисправных станков, а разные метрики просто по-разному читают один и тот же набор состояний.

Числовой checkpoint

Дальше в этом подпункте опираемся на точки r = 1, 10, 20, 33.

Они нужны не как “лучшие” значения, а как удобные опорные режимы: острая нехватка, промежуточный режим, почти снятая очередь и большой резерв.

Ниже в этом руководстве показан один опорный график этого подпункта, а остальные зависимости удобно читать по тем же checkpoint-значениям.

Блок Рекуррентное распределение состояний

Сначала задаём переходы между соседними состояниями через λ_i и μ_i.

Потом строим p_i шаг за шагом.

Так считать удобно, потому что количество неисправных станков меняется только на 1 вверх или вниз.

Нормировка закрывает всю цепь состояний от 0 до N.

Блок Простаивающие и ожидающие станки

По оси X меняется число наладчиков r.

По оси Y смотрим M_пр и M_ож.

Фиксированными остаются N = 36, Tc = 104 мин и Ts = 50 мин.

На checkpoint-точках получаем: при r = 1 имеем M_пр = 33.92, M_ож = 32.92; при r = 10 - 15.540 и 5.703; при r = 20 - 11.690 и 0.00206; при r = 33 - 11.688 и 7.686e-15.

По этим зависимостям видно, что сначала добавление наладчиков резко уменьшает ожидание.

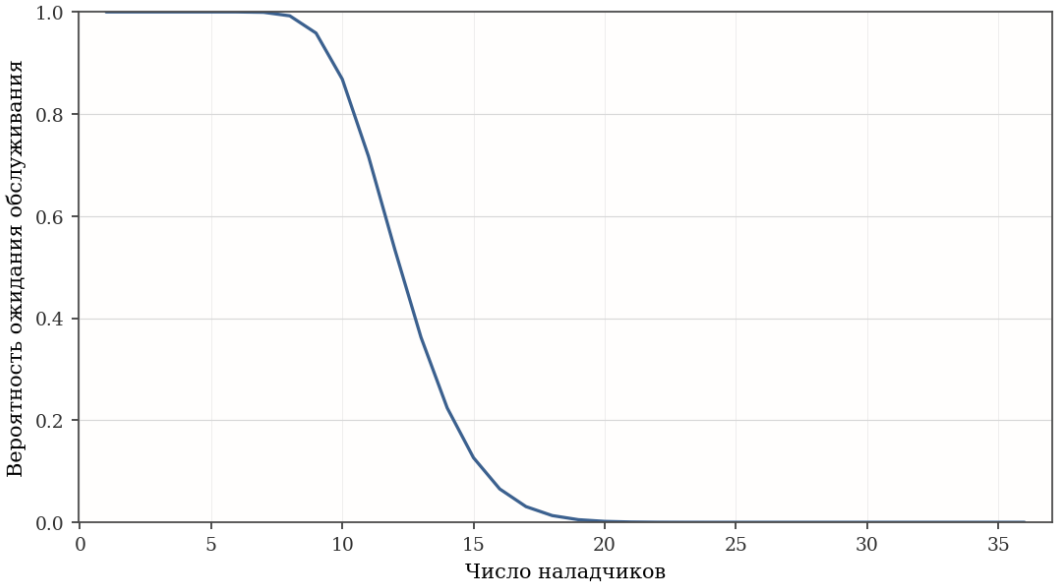
Потом очередь почти исчезает, и дальнейший рост r уже меняет в основном резерв, а не длину ожидания.

Первый безопасный checkpoint здесь - режим острой нехватки и первый режим, где очередь почти снята.

Безопасный вывод: сначала падает именно ожидание, а не все метрики одинаково и линейно.

Безопасный локальный вывод: этот набор зависимостей лучше всего защищает переход от явного дефицита наладчиков к режиму, где очередь уже почти снята.

Блок `Вероятность ожидания нового отказа`



Опорный график 2.1: вероятность ожидания обслуживания при увеличении числа наладчиков.

По оси `X` меняется `r`, а по оси `Y` стоит `P_ож`.

Эта метрика означает не “долю времени, когда очередь вообще есть”.

Она означает вероятность того, что именно новый отказавший станок попадёт в ожидание.

Это важное различие.

Здесь состояния нужно читать не просто по календарному времени, а с точки зрения нового отказа.

Поэтому для `P_ож` важно, из каких состояний ещё может прийти новый отказ и попадёт ли он при этом в режим, где все `r` наладчиков уже заняты.

Опасный вопрос здесь такой: почему `P_ож` нельзя заменить вероятностью наличия очереди?

Безопасный ответ: потому что вероятность наличия очереди говорит о текущем состоянии системы, а `P_ож` говорит о судьбе именно нового отказавшего станка.

По данным артефактов имеем: `P_ож = 1.000` при `r = 1`, `0.891` при `r = 10`, `0.00234` при `r = 20`, `2.109e-14` при `r = 33`.

Для сравнения, стационарная вероятность факта наличия очереди равна `1.000`, `0.876`, `0.00137`, `7.469e-15` в тех же точках.

Поэтому нельзя смешивать эти две вероятности в один и тот же смысл.

На защите лучше первым делом назвать, что это график про судьбу нового отказа, а не про календарное состояние очереди.

Безопасный локальный вывод: `P_ож` можно сравнивать с вероятностью наличия очереди только как две разные метрики, а не как одну и ту же кривую.

Блок `Занятые наладчики и их загрузка`

По оси `X` меняется `r`, а по оси `Y` стоят `M_зан` и `K_загр`.

На характерных точках имеем: при `r = 1` получаем `M_зан = 1.000`, `K_загр = 1.000`; при `r = 10` - `9.837` и `0.984`; при `r = 20` - `11.688` и `0.584`; при `r = 33` - `11.688` и `0.354`.

Здесь хорошо виден типичный компромисс.

Абсолютное число реально занятых наладчиков быстро выходит примерно к своему пределу.

Но относительная загрузка каждого наладчика падает, когда мы держим большой резерв.

Первый безопасный checkpoint здесь - момент, когда очередь уже почти снята, а `M_зан` перестаёт заметно расти.

По этой зависимости нельзя говорить, что дополнительные наладчики после насыщения “совсем не нужны”.

Безопасный вывод: после насыщения новый наладчик даёт уже в основном резерв, а не резкое уменьшение ожидания.

Локальный вывод

Для твоего варианта малое число наладчиков почти полностью переводит участок в режим ожидания.

Рост `r` сначала резко уменьшает очередь и простой.

Но после того как очередь почти исчезла, дальнейшее увеличение `r` в основном снижает загрузку самих наладчиков.

Значит, здесь есть явный компромисс между короткой очередью и эффективным использованием персонала.

Частая опасная путаница здесь такая: переносить сюда слова из первой задачи про очередь звонков и считать, что `P_ож`, `M_ож` и queue-метрики там означают одно и то же.

Безопасно говорить так: в `2.1` в очереди стоят не звонки, а неисправные станки, а `P_ож` относится к новому отказу станка; поэтому метрики из первой задачи можно сравнивать только по логике ожидания, но не смешивать по смыслу.

Что это значит простыми словами

Если наладчиков мало, сломанные станки просто стоят и ждут.

Если наладчиков становится больше, очередь быстро исчезает.

Но слишком большой запас означает, что часть наладчиков простаивает уже сама.

Поэтому нужно смотреть не только на отсутствие очереди, но и на цену такого резерва.

Если нужен ответ в одну-две фразы, удобно сказать так: при `r = 20` очередь почти исчезает, потому что `M_ож = 0.00206`, но `K_загр` уже падает до `0.584`.

Значит, после снятия очереди новые наладчики дают уже в основном резерв, а не большой выигрыш по ожиданию.

После этого лучше остановиться и не добавлять, что названная checkpoint-точка — это универсально лучшее число наладчиков.

Безопасная финальная точка такая: очередь почти снята, а дальше новые наладчики дают уже в основном резерв.

Что сказать на защите

Ниже четыре короткие карточки.

Карточка 1. Смысл модели

- Что могут спросить: Что означает состояние `S_i`?
- Короткий безопасный ответ: Это число неисправных станков. Если `i > r`, часть из них уже ждёт наладчика.
- Где чаще всего путаются: Думают, что `i` - это число занятых наладчиков.
- Каких слов лучше не говорить: “Это состояние с `i` занятыми наладчиками”.
- Один практический смысл: Модель считает не клиентов, а поломанные станки.

Карточка 2. Смысл метрики

- Что могут спросить: Чем `P_ож` отличается от вероятности наличия очереди?
- Короткий безопасный ответ: `P_ож` - это вероятность того, что новый отказавший станок попадёт в ожидание. Это не то же самое, что стационарная вероятность уже существующей очереди. При `r = 10` это `0.891` и `0.876`.
- Где чаще всего путаются: Сливают эти две вероятности в одну.
- Каких слов лучше не говорить: “Это просто вероятность того, что очередь есть”.
- Один практический смысл: Одна метрика про новое событие, другая про текущее состояние.

Карточка 3. Как читать график

- Что могут спросить: Как коротко читать графики по `r`?
- Короткий безопасный ответ: По оси `X` меняется число наладчиков `r`, а по оси `Y` смотрим простои, ожидание или загрузку. При росте `r` очередь сначала падает резко, а потом уже снижается в основном загрузка наладчиков.
- Где чаще всего путаются: Смотрят только на исчезновение очереди и забывают про цену резерва.
- Каких слов лучше не говорить: “Чем больше наладчиков, тем всегда лучше”.
- Один практический смысл: Нужно видеть и очередь, и цену большого резерва.

Карточка 4. Итоговый вывод

- Что могут спросить: Какой главный вывод по твоему варианту?
- Короткий безопасный ответ: При `r = 1` участок почти весь ждёт. К `r = 20` очередь почти исчезает: `M_ож = 0.00206`, но загрузка падает до `0.584`. При `r = 33` очередь почти нулевая, а `K_загр = 0.354`.
- Где чаще всего путаются: Считают максимальное `r` автоматически лучшим выбором.
- Каких слов лучше не говорить: “Надо просто взять как можно больше наладчиков”.
- Один практический смысл: После исчезновения очереди новый наладчик даёт уже в основном резерв, а не большой выигрыш по ожиданию.

Как использовать руководство на защите

Перед ответом удобно держать в голове только четыре шага.

Сначала сказать, что происходит в реальной системе.

Потом назвать, какая метрика ищется и что означает её символ.

Потом сослаться на один конкретный checkpoint своего варианта.

И только после этого сделать короткий практический вывод.